ALFRED

Новостная аналитика для бизнес-решений

Описание решения и соответствия требованиям хакатона

GitHub	github.com/dariaPchel/alfa-hack-2026
Репозиторий	Приватный; перед сдачей предоставить аккаунту жюри alfa-hack-2026 доступ на чтение.
Срез отчёта	26 октября 2026 года. Цифры по новостям — снимок на эту дату.
Версия кода	Текущая рабочая копия основана на HEAD 075bd6bd76e09d6f9ea814d90acf90fe05123704. Последующие локальные изменения ещё не входят в этот commit; перед отправкой зафиксируйте итоговый hash.

Кратко о решении

ALFRED объединяет обезличенный профиль компании, отраслевые новости и аналитические материалы Альфа-Банка. Пользователь выбирает компанию и событие, задаёт вопрос и получает раздельно показанные позицию банка, вывод модели, возможные риски и действия со ссылками на источники.

Это работающий прототип для предоставленного обезличенного набора. В документе отдельно отмечены ограничения: сервис пока не подключён к настоящему интернет-банку, не имеет пользовательской авторизации, а новости по обезличенным идентификаторам не подтверждают прямое упоминание конкретного клиента.

Пользовательский сценарий

- Открыть веб-интерфейс и выбрать компанию по обезличенному ID, отрасли или региону.
- Просмотреть новости компании/отрасли с датой публикации и ссылкой на источник.
- Выбрать новость, задать вопрос о влиянии на бизнес и дождаться анализа.
- Сопоставить позицию банка и вывод модели, открыть цитируемые источники и этапы обработки; оценить полезность ответа.

Архитектура и компоненты

React-интерфейс раздаётся через Nginx и обращается к Go API. PostgreSQL хранит профили, новости, фрагменты банковских документов, задания и результаты; Redis используется для кэша. Go API атомарно записывает задание с outbox, Kafka доставляет его отдельному Go analysis worker. Отдельный Go news worker получает новости через Serper. Необязательный research-профиль извлекает текст из PDF через Poppler/Tesseract, после чего Go importer сохраняет фрагменты в PostgreSQL.

React + Nginx	Go API	Kafka	Go workers
Интерфейс выбора и чата	Профили, запросы, API	Очередь анализа и outbox	Новости и анализ LLM
Браузер пользователя	PostgreSQL + Redis	Повторная доставка / DLQ	Serper + RouterAI + bank research

Контур анализа

- Go worker проверяет профиль, новость и URL источника, подбирает до пяти похожих компаний и до трёх релевантных фрагментов банковских материалов.
- Разрешённая LLM формирует план анализа, затем структурированный проект ответа с банковской позицией, выводом для компании, рисками, возможностями, действиями и source IDs.
- Отдельный вызов-критик проверяет полноту и подтверждённость источниками. При замечании worker ограниченно повторяет формирование проекта; Kafka даёт до трёх попыток обработки задания.
- Результат хранит ссылки и короткий trace. Повторная доставка завершённой задачи не должна создавать дубликат результата.

RouterAI использует alias `~z-ai/glm-flash-latest` и канонический ID `z-ai/glm-5.3-flash`. Первоначальный HTTP 400 вызвало неверное имя модели. Затем RouterAI ответил HTTP 200, но весь лимит 1 800 токенов ушёл в reasoning (`finish_reason=length`), без текста для пользователя. Go worker теперь отправляет `max_tokens=32000` и использует стандартную сетевую маршрутизацию; внутренних лимитов числа LLM-запросов и расхода нет. После перезапуска worker выполнен один короткий прямой запрос: HTTP 200, модель `z-ai/glm-5.3-flash`, текст `OK`, `finish_reason=stop`.

Данные и результаты прототипа

В локальной базе на момент последней фиксации README указаны 85 513 обезличенных профилей компаний и 177 фрагментов банковских материалов из девяти PDF. Для первых 20 компаний зафиксирован снимок 211 отраслевых новостей за 90 дней, включая 100 за последние семь дней. Источники и объёмы могут измениться при обновлении Serper; перед демонстрацией их следует пересчитать.

- Профили, связи физических лиц и агрегированные платежи загружаются из файлов кейса. Персональные и юридические идентификаторы обезличены.
- Публикации Serper показываются с датой, URL и коротким поисковым фрагментом. Полный текст статьи не скачивается; прямое упоминание обезличенной компании автоматически не подтверждается.
- Банковские выдержки выбираются по совпадению отраслевых, новостных и тематических слов, а не через полноценный embedding/vector RAG в текущем Go-контуре.
- Демо-компания U001 и её новости синтетические и включаются только при `DEMO_SEED=true`. Для рабочего сценария с данными Т3 используется `DEMO_SEED=false`.

Сверка с критериями ТЗ

Критерий	Статус	Что реализовано / ограничение
Персональный бизнес-контекст и отбор событий	Частично	Учитываются отрасль/ОКВЭД, регион, выручка, численность и похожие компании. Для обезличенных ID поиск отраслевой, поэтому прямую новость конкретного клиента подтвердить нельзя.
Позиция Альфа-Банка и раздельная атрибуция выводов	Частично	Позиция банка и анализ модели выводятся раздельно; к ответу прикладываются банковские выдержки. Глубина покрытия зависит от наличия релевантного фрагмента среди импортированных документов.
Первоисточники и фактологическая проверка	Частично	Хранятся ссылки, дата, цитаты и source IDs; критик проверяет заявленные источники. Текст статьи не извлекается, поэтому проверить все факты первоисточника нельзя.
Содержательное влияние, риски, возможности и действия	Реализовано в потоке	Схема ответа требует bank view, company analysis, risks, opportunities и solutions; вывод должен различать факт и интерпретацию.
Многошаговая агентная логика	Частично	Есть планирование, генерация и отдельная проверка-критик с ограниченной коррекцией. Нет общего набора динамических веб-инструментов или автономного поиска до закрытия пробелов.
Трассируемость и уточнения пользователя	Частично	UI показывает citations и этапы trace и принимает вопрос/оценку. Ответы не формируют полноценную постоянную историю пользовательских диалогов.
Устойчивость на ранее неизвестных событиях	Не подтверждено	News worker регулярно получает реальные новые результаты Serper, но систематическая оценка точности на отложенной выборке и экспертная разметка ещё не проведены.
Масштабирование и сопровождение	Частично	API, news worker и analysis worker разделены; Kafka, PostgreSQL и Redis предусмотрены. Локальный Compose — одноброкерный демонстрационный профиль, production sizing/HA не настроены.
Безопасность AI и клиентских данных	Частично	Есть allowlist LLM, обезличенный набор, хранение ключей в локальном `.env`, обработка входных данных как недоверенных. Веб-интерфейс не имеет полноценной аутентификации и разграничения доступа.
Полный сценарий через Альфа-Бизнес	Частично	Прототип показывает сценарий в отдельном веб-интерфейсе. Встраивания в интернет-банк, реальной клиентской авторизации и подключения к production API банка нет.

Запуск и проверка

Требования: Docker Desktop с Compose, доступ к портам проекта, локальные обезличенные CSV и банковские PDF из `docs/`, а для реальных новостей и анализа — ключи Serper и RouterAI. Секреты хранить только в `.env`, не в репозитории.

- Из корня репозитория создать `.env` из `.env.example`; проверить пути к трём CSV, `DATASET_KIND=anonymized`, `DEMO_SEED=false`, RouterAI base URL, alias `~z-ai/glm-flash-latest`, обычную сетевую маршрутизацию хоста до RouterAI; внутренних лимитов числа запросов или бюджета LLM нет.
- Запустить воспроизводимый Compose-контур с исследовательским профилем: `docker compose -f compose.p1.yaml --profile research up --build`. Профиль research извлекает страницы PDF и импортирует банковские фрагменты; первичный OCR может занять время.
- Открыть `http://localhost:5173`, выбрать одну из компаний с новостями, открыть публикацию, задать вопрос и проверить ответ, источники, trace и форму обратной связи.
- Проверить состояние компонентов и ошибки: `docker compose -f compose.p1.yaml ps` и `docker compose -f compose.p1.yaml logs importer research-exporter research-importer news-worker worker api`.
- Проверить историю задач/качество через `GET /api/quality`; повторную отправку задачи выполнять только по инструкции README и с известным analysis ID.

Поставка в репозитории

- Единый запуск Compose описан в корневом README; Go зависимости закреплены в `go.mod`/`go.sum`, frontend — в `package-lock.json`.
- `THIRD_PARTY_NOTICES.md` описывает библиотеки, контейнеры, RouterAI, Serper, входной набор и синтетические данные.
- Тесты, lint/build и команды сквозной проверки перечислены в README; результаты запусков нужно повторить на финальном commit перед сдачей.
- Private origin: `https://github.com/dariaPchel/alfa-hack-2026`. По ТЗ нужно дать аккаунту жюри `alfa-hack-2026` доступ к репозиторию и указать финальный commit hash или tag. На момент подготовки документа текущая рабочая копия содержит изменения поверх указанного HEAD.

План доведения до требований

- Заменить отраслевую выдачу на проверку релевантности конкретному клиенту с учетом группы компаний, регионов, контрагентов и платежных связей; измерять precision/recall на экспертной выборке.
- Получать и хранить текст или надёжные выдержки первоисточника с правами доступа; проверять даты и утверждения на уровне отдельных фактов.
- Расширить retrieval по банковским материалам и показывать точную связь каждого тезиса с документом/страницей.
- Добавить тестовый набор ранее не показанных событий, регулярную экспертную оценку и мониторинг качества персонализации/LLM.
- До production-подключения добавить аутентификацию, авторизацию по клиенту, управление секретами, аудит доступа и высокодоступную конфигурацию брокера/БД.

Основание: техническое задание «Новостная аналитика для принятия бизнес-решений», прежде всего разделы «Требования к агентной логике», «Формат сдачи», «Критерии успеха», «Ограничения» и приложение со списком разрешённых LLM. Фактические сведения о реализации сверены с README и исходным кодом этого репозитория.

Актуализация реализации

Состояние на 26 октября 2026 года · дополнение к отчёту на страницах 2–3

Активный путь анализа

Go worker передаёт один запрос в RouterAI: профиль компании, аналоги, новость, вопрос пользователя и отобранные банковские выдержки. В system prompt задана JSON-структура из шести полей: bank_view, ai_analysis, risks, opportunities, solutions и source_ids.

Go проверяет обязательные поля, массивы и каждый ID источника, собирает ссылки для UI и сохраняет trace. Для активного запроса передаются max_tokens=32000 и reasoning.effort=low. Исходный таймаут оставлен равным 50 секундам.

Прежний многовызовный сценарий с планом, ответом и отдельной проверкой сохранён в generateAnalysisWithExperts, но отключён через expertAnalysisWorkflowEnabled = false. Описание этого сценария на страницах 2–3 выше заменяется текущим дополнением.

Проверка в приложении

Шаг	Результат
Сборка и запуск worker	Go worker собран и перезапущен через Compose.
Короткий запрос RouterAI	HTTP 200; модель разрешилась как z-ai/glm-5.3-flash; ответ OK, finish_reason=stop.
Анализ из очереди	Задача завершилась со статусом complete, provenance=approved-llm; trace подтверждает один структурированный вызов и четыре цитаты.
Маршрутизация и лимиты приложения	Wi-Fi proxy удалён. Запросы идут стандартным сетевым путём. Внутреннего лимита числа вызовов или бюджета LLM нет.

Это дополнение фиксирует одновызовную реализацию и фактическую проверку. Оно не меняет остальные данные, критерии и ограничения решения, перечисленные в основном отчёте.